Assignment 1: Pentium M Hybrid Branch Predictor Report

Oyekunle Oshidele - 2143251

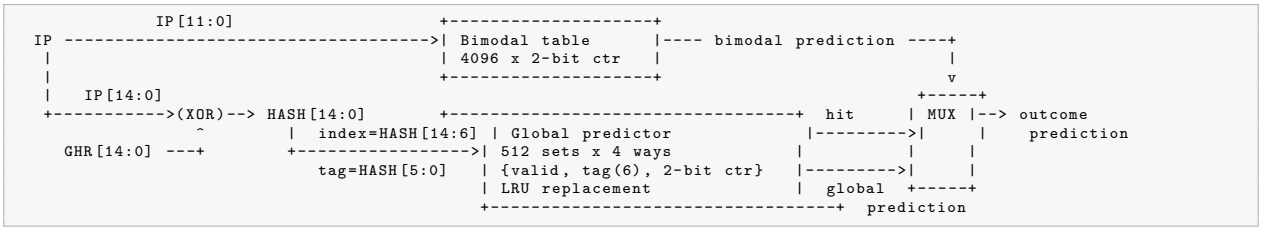
October 20, 2026

1. Objective

The goal was to implement the branch outcome (taken / not-taken) prediction part of the Pentium M branch prediction unit in the CBP2 simulation infrastructure, and to evaluate it on the 20 supplied traces. As the assignment specifies, the implementation covers the **bimodal table** and the **global predictor** (the blue blocks in the Pentium M diagram). The loop predictor, the path information register (PIR) hash and the branch target prediction (BTB / iBTB) are not implemented.

2. Design

The structure follows the reverse-engineered Pentium M organisation (Uzelac & Milenković, ISPASS 2009) and the hybrid predictor example in Appendix B of the assignment.



Component	Organisation	Storage
Bimodal table	4096 × 2-bit saturating counters, index `IP[11:0]`	8 Kbit
Global predictor	512 sets × 4 ways; entry = valid (1) + tag (6) + 2-bit counter; 2-bit LRU rank per entry	2048 × 11 bit = 22 Kbit
Global history register	15 bits (outcomes of conditional branches)	15 bit
Total		**≈ 30 Kbit (3.75 KB)**

The 2-way variant uses 512 × 2 × 10 bits = 10 Kbit for the global table (18 Kbit in total). For comparison, the gshare predictor supplied with the infrastructure uses 32K × 2 bits = 64 Kbit (8 KB).

2.1 Prediction (`pm_predictor::predict`)

For a conditional branch:

1. **Bimodal lookup.** `index = IP & 0xFFF`. The bimodal prediction is the counter's most significant bit (taken when the counter is 2 or 3).
2. **Global lookup.** `HASH = (IP XOR GHR) & 0x7FFF`, `set = HASH[14:6]`, `tag = HASH[5:0]`. All ways of the set are searched for a valid entry with a matching tag.
3. **Selection.** On a global hit, the global entry's counter gives the final prediction. Otherwise the bimodal prediction is used. This is the "hit" multiplexer in the Pentium M diagram.

The update object (`pm_update`) records the bimodal index, the set, the tag, the hitting way and the bimodal prediction, so `update()` does not recompute them.

Unconditional branches (calls, returns, jumps) are always predicted taken. Target prediction always returns 0 because the assignment does not require it.

2.2 Update (`pm_predictor::update`)

For a conditional branch:

1. The **bimodal counter** is always trained with the outcome, which matches the Appendix B example.
2. **On a global hit**, the hitting entry's counter is trained, and the entry becomes the most recently used (MRU) entry of its set.
3. **On a global miss**, a new entry is allocated **only if the bimodal prediction was wrong**. This rule follows the Pentium M (and generally tagged-hybrid) policy: branches that the bimodal table already predicts correctly do not need a global entry. The victim is an invalid way if one exists, otherwise the LRU way. The new entry's counter is set to the weak state in the direction of the outcome (2 for taken, 1 for not-taken).
4. The outcome is shifted into the 15-bit **global history register**.

LRU is implemented with a recency rank (0 = MRU ... ways-1 = LRU) per entry. When an entry is touched, every entry with a smaller rank is incremented, and the touched entry gets rank 0. The cache structure and replacement logic were written from scratch, so no external cache code was used.

2.3 Simplifications relative to the real Pentium M

- The PIR (path information register, which hashes the addresses of taken branches) is replaced by a plain global history register of conditional branch outcomes, as the assignment allows. The XOR hash width (15 bits) is kept.
- The loop predictor and the BTB/iBTB are not modelled.
- Both the 4-way (Pentium M) and 2-way (allowed by the assignment) global predictors are supported through the `PM_GLOBAL_WAYS` macro. The default is 4-way.

3. Methodology

The 20 CBP2 traces were run (each trace represents 100 M instructions). The metric is **mispredictions per kilo-instruction (MPKI)** for conditional branches, as reported by `predict`. `experiments.sh` built and ran 7 configurations:

Name	Description
`always_taken`	Static predict-taken (the original stub)
`gshare_32K`	Supplied gshare: 32K counters, 15-bit history (64 Kbit)
`bimodal_4K`	Bimodal table alone (global predictor disabled)
`pm_2way_alloc_always`	Hybrid, 2-way global, allocates on every global miss
`pm_2way`	Hybrid, 2-way global, allocates only on a bimodal mispredict
`pm_4way_alloc_always`	Hybrid, 4-way global, allocates on every global miss
`pm_4way`	**Hybrid, 4-way global, allocates only on a bimodal mispredict (submitted default)**

All runs used g++ 9.4 with `-O3` on Ubuntu 20.04 (WSL).

4. Results (MPKI, lower is better)

Trace	always taken	gshare 32K	bimodal 4K	PM 2-way (alloc always)	PM 2-way	PM 4-way (alloc always)	PM 4-way
164.gzip	70.025	12.473	15.030	12.617	12.520	12.459	**12.436**
175.vpr	85.312	13.415	14.671	14.337	13.866	14.070	**13.298**
176.gcc	86.071	11.254	19.347	15.298	10.618	13.304	**9.285**
181.mcf	102.497	15.837	33.245	17.680	16.398	16.488	**15.621**
186.crafty	48.651	5.837	13.075	8.464	5.983	6.895	**5.211**
197.parser	78.421	10.008	15.417	10.357	9.036	9.242	**8.538**
201.compress	76.983	7.831	9.356	8.199	7.854	7.839	**7.702**
202.jess	92.343	1.562	8.809	1.750	1.149	1.464	**1.118**
205.raytrace	109.506	2.756	3.806	3.421	2.503	2.952	**2.307**
209.db	91.451	3.909	6.480	4.613	3.723	4.283	**3.615**
213.javac	109.644	2.267	3.181	2.638	2.126	2.397	**2.037**
222.mpegaudio	110.339	2.188	3.154	2.604	2.124	2.352	**2.028**
227.mtrt	112.022	2.657	3.706	3.268	2.470	2.957	**2.383**
228.jack	77.958	3.033	7.691	3.946	2.420	3.075	**2.353**
252.eon	24.226	1.807	9.405	3.139	1.501	1.837	**1.425**
253.perlbmk	66.833	2.554	5.955	4.700	1.917	3.279	**1.819**
254.gap	75.950	3.926	9.675	4.797	3.604	3.814	**3.507**
255.vortex	70.524	1.222	1.916	1.239	0.854	1.085	**0.828**
256.bzip2	178.257	0.094	0.112	0.076	0.059	0.073	**0.057**
300.twolf	59.221	21.489	21.942	22.046	21.981	22.236	**21.389**
Average	**86.312**	**6.306**	**10.299**	**7.259**	**6.135**	**6.605**	**5.848**

Storage: gshare 64 Kbit; bimodal 8 Kbit; PM 2-way ≈ 18 Kbit; PM 4-way ≈ 30 Kbit.

5. Discussion

1. **The hybrid predictor works.** Adding the 4-way global predictor to the 4K bimodal table lowers average MPKI from 10.30 to 5.85, a **43 % reduction**. The largest gains are on control-intensive integer codes such as `176.gcc` (19.3 → 9.3), `181.mcf` (33.2 → 15.6), `186.crafty` (13.1 → 5.2) and `252.eon` (9.4 → 1.4). These branches are correlated with the outcomes of preceding branches, which the global history captures and a per-address counter cannot.

2. **It beats a larger gshare.** The Pentium M style predictor (≈ 30 Kbit) does better than the supplied 64 Kbit gshare on **all 20 traces** (5.85 vs 6.31 MPKI on average, 7.3 % fewer mispredictions) with less than half the storage. Even the 2-way version (≈ 18 Kbit, 6.14 MPKI) beats gshare. There are two reasons. First, the tags prevent destructive aliasing in the global table. Second, the global table only holds the branches that need history, while easy branches stay in the cheap bimodal table.

3. **The allocation policy matters most.** Allocating a global entry on every miss (`alloc_always`) costs 13–18 % more mispredictions than allocating only when the bimodal table mispredicts (2-way: 7.26 vs 6.14; 4-way: 6.61 vs 5.85). When every branch is inserted, the many easy, strongly biased branches evict the useful history-dependent entries. On `176.gcc`, which has a large branch footprint, the difference is 13.3 vs 9.3 MPKI. Filtering allocation through the bimodal predictor is what makes a small tagged table effective.

4. **4-way beats 2-way.** With the same number of sets, 4 ways reduce MPKI from 6.14 to 5.85 (−4.7 %). The gain comes from fewer conflict misses, and it is largest on the benchmarks with the most branches (`gcc` −12.6 %, `crafty` −12.9 %, `vpr` −4.1 %). The extra storage (about 12 Kbit) gives a moderate gain. The assignment allows the 2-way design as a cheaper alternative.

5. **Remaining mispredictions.** `300.twolf`, `181.mcf`, `175.vpr` and `164.gzip` stay above 12 MPKI for every predictor tested, including gshare. Their branches depend mainly on data values (comparisons in placement/annealing, pointer chasing and compression loops) rather than on branch history, so neither component of the hybrid can learn them. The loop predictor of the full Pentium M design targets loops with fixed trip counts, so it would probably recover only part of these mispredictions. This was not measured.

6. Conclusion

A Pentium M style hybrid predictor was implemented: a 4K-entry bimodal table plus a 512-set, 4-way, 6-bit-tagged global predictor indexed by `IP XOR GHR`, with the global prediction used on a hit. It reaches an average of **5.848 MPKI** on the 20 CBP2 traces. This is 93 % fewer mispredictions than static predict-taken and 43 % fewer than the bimodal table alone, and it is better than a 64 Kbit gshare while using less than half its storage. The experiments show that the bimodal-mispredict allocation filter and the global-table associativity are the most important design choices.